001

Dore's Pub
Development and Design Document

Daniel George Mark Dore

March 2023 - Present

Brief

This is the design and development document for my current project - Dore's Pub. Taking inspiration from games such as the Sims series and Project Architect, Dore's Pub allows the player to create and run their own pub or bar. With some experience in the industry and with a friend as a Shift Manager working at one of Norwich's busiest Wetherspoon's, the game will give the player a good experience of the ups and downs of running a pub, while also giving them good creative freedom to design it too their will.

Contents

Design **3**

Development **3**

1 Editor Character . . . 3

2 Parent Tools . . . 3

3 Basic UI . . . 4

4 Changing Movement Speeds . . . 5

5 Item (Tool) Data Library . . . 6

6 Data Table Object . . . 6

7 Building Manager . . . 7

8 Select Tool . . . 7

9 Item Tool UI . . . 8

10 Item Tool Functionality . . . 8

11 Building Storage . . . 9

12 Building Grid Ver 1 . . . 10

List of Figures

1 The EditorCharacter 3
2 Player Held Code Graph 4
3 Player to Tool Graph 4
4 Component-izing UI 4
5 Tool Select Components 5
6 Button Interaction 6
7 The Data Table 7
8 The CatButton 8
9 The FurnishingBar 8
10 The AddFoundItem macro 8
11 Item UI In Game 8
12 PlacerMesh in use 9
13 Placed Furnishings 9

Design

Development

1 Editor Character

I started by creating the editor character and the basic movement properties that the player would use to move the camera around. The character is simply a ACharacter class with a camera attached to a spring arm, rotated so the camera faces the ground at an angle.

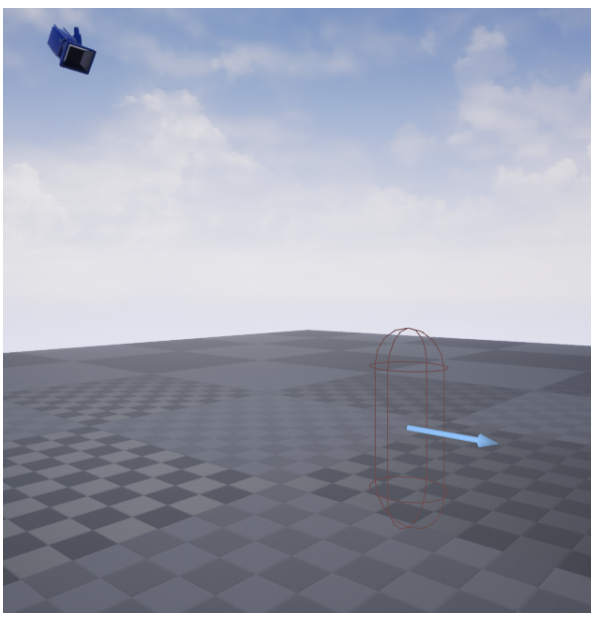

Figure 1. Example of the EditorCharacter

To make sure the character doesn't move around without the player pressing inputs, I made a helper function called GetHasAxisValue(), which will check if there is any actual axis input. Usually I would create a macro to fill this role, but a function works in a similar way.

```
void AEditorPlayer::MoveX(float AxisValue)
{
if (GetHasAxisValue(AxisValue))
{
        AddActorLocalOffset(FVector((MovementStep * AxisValue),
↪  0.0f, 0.0f), false, false);
}
}
```

```
void AEditorPlayer::MoveY(float AxisValue)
{
if (GetHasAxisValue(AxisValue))
{
        AddActorLocalOffset(FVector(0.0f, (MovementStep *
↪  AxisValue), 0.0f), false, false);
}
}

void AEditorPlayer::RotateX(float AxisValue)
{
if (GetHasAxisValue(AxisValue))
{
        AddActorWorldRotation(FRotator(0.0f, (RotationDegree *
↪  AxisValue), 0.0f), false, false);
}
}
```

Zooming the camera is slightly different, as there is a min and max bounds the player can move between (we don't want the player to zoom too far out they can't see their pub, no?). I use two float values called MinZoomValue and MaxZoomValue, with a ZoomMultiplier controlling how fast they change the spring arm's length.

```
void AEditorPlayer::ZoomCamera(float AxisValue)
{
if (GetHasAxisValue(AxisValue))
{
        if (AxisValue == 1)
        {
                if (PlayerCameraSpringArm->TargetArmLength +
↪  ZoomMultiplier >= MaxZoomValue)
                {
                        PlayerCameraSpringArm->TargetArmLength
↪  = MaxZoomValue;
                }
                else
                {
                        PlayerCameraSpringArm->TargetArmLength
↪  = PlayerCameraSpringArm->TargetArmLength + ZoomMultiplier;
                }
        }
        else if (AxisValue == -1)
        {
                if (PlayerCameraSpringArm->TargetArmLength -
↪  ZoomMultiplier <= MinZoomValue)
                {
                        PlayerCameraSpringArm->TargetArmLength
↪  = MinZoomValue;
                }
                else
                {
                        PlayerCameraSpringArm->TargetArmLength
↪  = PlayerCameraSpringArm->TargetArmLength - ZoomMultiplier;
                }
        }
}
}
```

2 Parent Tools

Next, I worked on how the player would interact with their level, through tools. There was multiple ways that I could use to allow the player to use these tools, but making sure the player only used the tool they had selected was top priority. Originally, I thought about having all of the tool code in the EditorPlayer class itself, but this would require multiple different functions and a switch on an enum to work correctly.

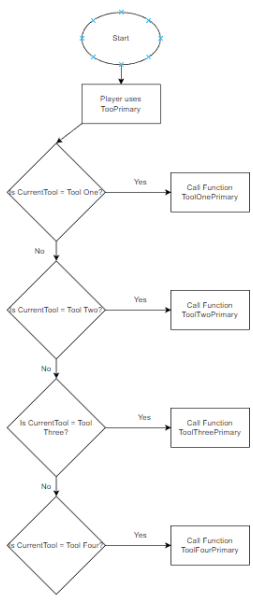

Figure 2. Interaction between using Tools in the EditorPlayer class

To fix this issue, I used ActorComponents to separate each tool's code into different classes, with a map and enum used to select the correct tool. Any code used by multiple tools can be added to the ParentTool class, rather than repeating them in their own tool functions. Additionally, this also meant that tools could be locked off from the player (such as a tutorial mode) by simply not adding them to the player.

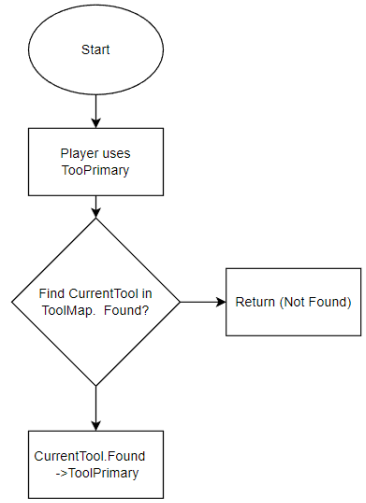

Figure 3. Interaction between using Tools as separate ActorComponents

I also created a ToolDataLibrary to store any enums or structs used by tools or by classes to use tools. Currently, only an enum exists in this class - EToolType.

```
UENUM()
enum EToolType
{
        Explore UMETA(Display Name = "Explore"),
        Select UMETA(Display Name = "Select"),
        Build UMETA(Display Name = "Build"),
        Item UMETA(Display Name = "Item"),
};
```

3 Basic UI

I worked on making a basic frame for the UI next, as most of the game would be interacted through the UI itself. Using framework I had created before, I component-ized the UI into one main Widget, multiple UI states and then components that would be present on these states.

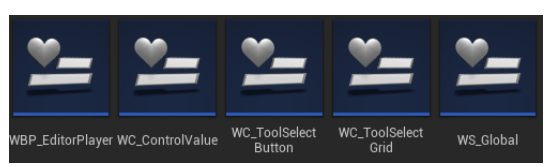

Figure 4. Component-izing my UI

The main widget features an overlay that has the GlobalState (components that will be visible on every state) and a widget switcher that is used to swap between states. To allow interaction between my c++ classes and the UI, I created a parent c++ class that only featured a widget switcher bind.

```
public:
UPROPERTY(BlueprintReadOnly, meta = (BindWidget))
        class UWidgetSwitcher* ToolState = nullptr;
```

To allow the player to switch tools through the UI, I created two components - ToolSelectButton and ToolSelectGrid. Each button is simply an overlay of an image, text block and invisible button, while the grid is a Uniform Grid Panel with four ToolSelectButtons.

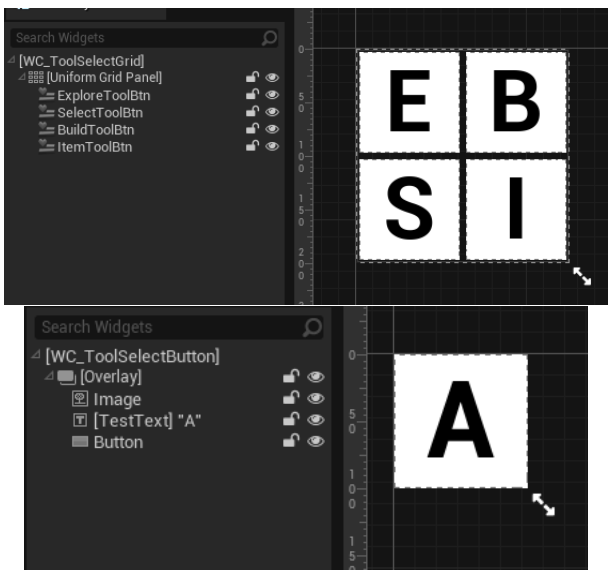

Figure 5. The hierarchy of the Tool Select Components

The button's interaction will be added on the Widget Blueprint as the blueprint is the only class with interaction back to the player. There are other options to implementing the UI interaction, such as adding a pointer to the player on each state, but as this is the first UI passover I have added the interaction on the main blueprint.

4 Changing Movement Speeds

Continuing the basic UI, I added two components that allow the player to change their rotation and movement speed between different bounds. I added four new float values for the bounds in the EditorPlayer class and two functions to modify the speed value.

```
UPROPERTY(VisibleAnywhere, BlueprintReadWrite, Category =
↪  "Control Properties")
float RotationMin = 1;

UPROPERTY(VisibleAnywhere, BlueprintReadWrite, Category =
↪  "Control Properties")
float RotationMax = 5;

UPROPERTY(VisibleAnywhere, BlueprintReadWrite, Category =
↪  "Control Properties")
float MovementMin = 2;

UPROPERTY(VisibleAnywhere, BlueprintReadWrite, Category =
↪  "Control Properties")
float MovementMax = 10;

void AEditorPlayer::ModifyRotationDegree(bool bIncrease)
{
if (bIncrease == true) {
        if (RotationDegree + 1 >= RotationMax) {
                RotationDegree = RotationMax;
        }
        else {
                RotationDegree++;
        }
}
else {
        if (RotationDegree - 1 <= RotationMin) {
                RotationDegree = RotationMin;
        }
        else {
                RotationDegree--;
        }
}
}
```

Finally, I added UI elements to control the speed of the movement and rotation and implemented the buttons in the EditorUI widget blueprint.

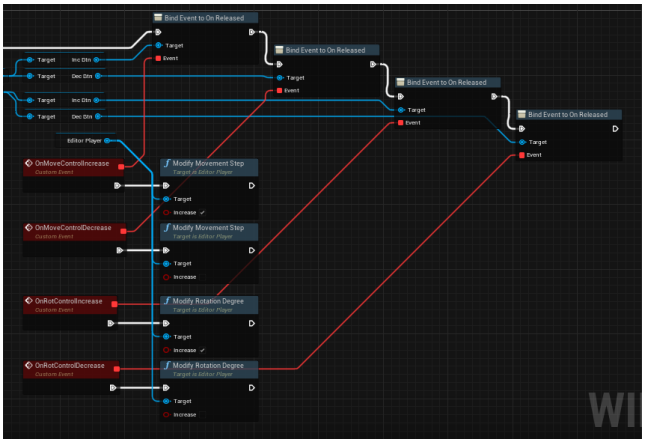

Figure 6. How the button interaction is implemented

5 Item (Tool) Data Library

Usually, I have all of my user created structs and enums in one data library. This time I have separated them into separate classes based on what each class needs to reduce unnecessary inclusions. For all data structures required by thee furnishings, I created a new BlueprintFunctionLibrary - ItemDataLibrary.

I added two enums to start, both used for defining a furnishings's category and sub category. These will be used by a later UI object to search via selecting categories. During enum creation, I had both default types named DefaultType, but this created compiler errors.

```
UENUM()
enum EFurnishingType
{
        DefaultType UMETA(Display Name = "Default"),
        Seating UMETA(Display Name = "Seating"),
};

UENUM()
enum EFurnishingSubType
{
        DefaultSubType UMETA(Display Name = "Default"),
        Seating_Sofa UMETA(Display Name = "Seating Sofa"),
        Seating_Bench UMETA(Display Name = "Seating Bench"),
};
```

Following, I added a struct that extends from FTableRowBase, allowing me to create a data table to store the information about each furnishing object in the world. This data will also be stored in the furnishing itself, allowing the Select tool to identify what the furnishing is.

```
USTRUCT(BlueprintType)
struct FFurnishingItem : public FTableRowBase
{
        GENERATED_USTRUCT_BODY();

public:
        UPROPERTY(EditAnywhere, BlueprintReadWrite)
                FString Name;

        UPROPERTY(EditAnywhere, BlueprintReadWrite)
                FString Description;

        UPROPERTY(EditAnywhere, BlueprintReadWrite)
                TEnumAsByte<EFurnishingType> Category;

        UPROPERTY(EditAnywhere, BlueprintReadWrite)
                TEnumAsByte<EFurnishingSubType> SubCategory;

        // Class
        UPROPERTY(EditAnywhere, BlueprintReadWrite)
                TSubclassOf<class AParentFurnishing> Class;

        UPROPERTY(EditAnywhere, BlueprintReadWrite)
                UTexture2D* Icon;

        UPROPERTY(EditAnywhere, BlueprintReadWrite)
                UStaticMesh* Mesh;

        UPROPERTY(EditAnywhere, BlueprintReadWrite)
                float Price;
};
```

Finally, I added a new Actor class which is used for all place-able furnishings - ParentFurnishing. Currently, this class only contains mesh components and the data structure I created earlier. Anything that will be used by multiple or all of the furnishing types will be added to this class instead of adding it to classes individually.

6 Data Table Object

Following the previous section, I created the data table object. These can be created from a cvar or Excel spreadsheet file, but I tend to create them in the editor themselves. For this stage of development, I only have added all of the 'Green Sofa' props to the data table, but more will be added later.

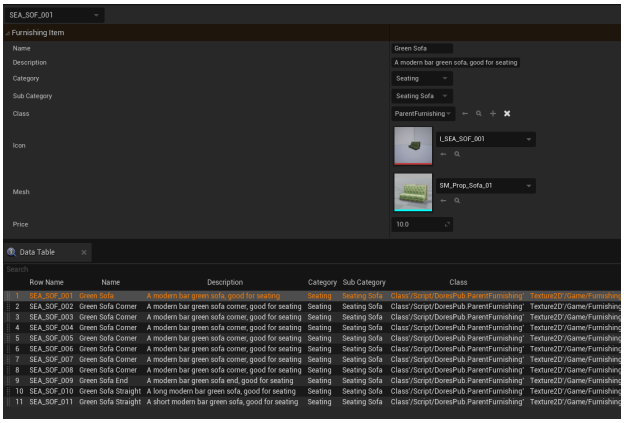

Figure 7. The data table with all the 'Green Sofa' props

7 Building Manager

The BuildingManager was the next class that I implemented, which will be used whenever the building itself is modified. It will be used to store all the info that corrisponds with the building itself, such as placed furnishings, walls, floor and ceilings, as well as the grid and size of the world which can be used by the player.

I started by finding and storing the data table I created last section. I also implemented a function that will be used to sort through the data table, find a matching row via name, spawn the found class at a location in the world, set it's mesh and finally store the new object in the world array. This function returns a bool, which will allow the function to break if we don't find a row from a matching ID.

```
bool ABuildingManager::SpawnFurnishingAtLocation(FName
↪  FurnishingID, FVector WorldLocation, FRotator
↪  WorldRotation)
{
        // Get Object from Data Table
        FFurnishingItem* item =
↪  FurnishingDataTable->FindRow<FFurnishingItem>(FurnishingID,
↪  "", false);
        // Check if we found an item, if we did, continue...
        if (item == nullptr) { return false; }

        // Spawn data for the furnishing
        FActorSpawnParameters furnishingSpawnParams;
        furnishingSpawnParams.SpawnCollisionHandlingOverride =
↪  ESpawnActorCollisionHandlingMethod::AdjustIfPossibleBut...;

        // Spawn the item in the world
        AParentFurnishing* newFurnishing =
↪  GetWorld()->SpawnActor<AParentFurnishing>(item->Class,
↪  WorldLocation, WorldRotation, furnishingSpawnParams);
        newFurnishing->Mesh->SetStaticMesh(item->Mesh);

        // Set the new objects data
        // TODO - Find a better way of doing this
        newFurnishing->Data.Name = item->Name;
        newFurnishing->Data.Description = item->Description;
        newFurnishing->Data.Category = item->Category;
        newFurnishing->Data.SubCategory = item->SubCategory;
        newFurnishing->Data.Icon = item->Icon;
        newFurnishing->Data.Price = item->Price;

        // However, set the data not needed to nullptr
        // This is stored somewhere else in the object (class is
↪  itself, mesh on Mesh component)
        newFurnishing->Data.Mesh = nullptr;
        newFurnishing->Data.Class = nullptr;

        return true;
}
```

8 Select Tool

The first tool, other than the Explore tool that has no interaction, that I implemented was the Select tool. This tool handles any interaction between the EditorPlayer and placed furnishings, which includes: selling placed furnishings, modifying the storage of containers and returning the data of the furnishing.

Firstly, I added the ActorComponent to the EditorPlayer and added it to the ToolMap. However, while testing ToolPrimary on the Select tool, it would not get called correctly and would call ParentTool-¿ToolPrimary instead. To work around this error, I implemented an interface and added it to each of the tools. I also created a furnishing interface, which will be used later.

Finally, I implemented the basic state of the Select Tool. Currently, it fires a trace where the mouse is on the screen - through GetHitRestultUnderCursor - and Casts to the Furnishing Interface. If the class hit has the interface, it calls it's version of GetObjectData that simply returns the name of the furnishing.

```
void USelectTool::ToolPrimary() {
UE_LOG(LogTemp, Warning, TEXT("Select Tool Primary"));

// Get object under mouse
ToolOwner->PC->GetHitResultUnderCursorByChannel(TraceChannel,
↪  false, TraceHit);
IFurnishingInterface* inf =
↪  Cast<IFurnishingInterface>(TraceHit.Actor);
if (inf) {
        FString outName = inf->GetObjectData();
```

```
        UE_LOG(LogTemp, Warning, TEXT("%s"), *outName);
}
```

9 Item Tool UI

Before creating the functionality of the Item Tool, I created the UI for the Item State. Currently, this state only consists of four widget and three object classes, but will use more later - for selecting delete mode for example.

Both SectionButton and SubSectionButton are used for selecting the section of the data table to display. They are an overlay of Image, Text and Button components and are used in the FurnishingBar. They both inherit the UserObjectListEntry interface so they can be used in lists. Their corresponding data object only contain a struct and a pointer to the owning FurnishingBar.

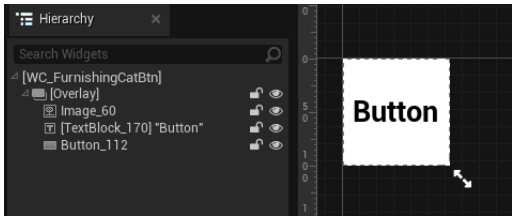

Figure 8. The hierarchy of the CategoryButton Component

The FurnishingBar is used to display the found furnishings from the selected section and subsection. It also sorts out the select-able subsections based on section they chose, rendering non-matching queries unselectable - for example, players searching for seating will only see seating subsections and not table subsections. Edge cases are also handled, such as if no section is selected, all items are displayed.

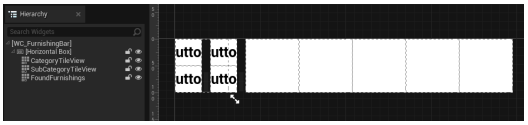

Figure 9. The hierarchy of the FurnishingBar Component

Once a new search term is chosen, the bar is cleared and all found furnishings are added to the List View widget through a macro, which displays individual buttons with icons corresponding to the furnishing in the button.

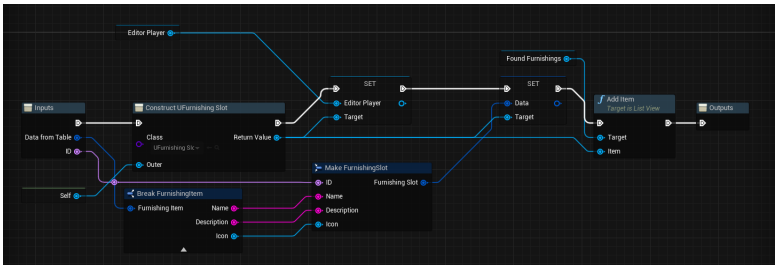

Figure 10. A macro for found items to the list of the FurinshingBar

Figure 11. How the Item UI looks in game

10 Item Tool Functionality

With the UI functionality added, I moved on to the Item Tool itself. I started by adding a new mesh to the EditorPlayer, which will be used to display where the selected furnishing will be placed in the world based on the mouse location. Originally, I experimented around adding the component in real-time when an item is selected in the tool, but I couldn't attach the component to the player and would not spawn at all due to ActorComponents not having components themselves.

```
EditorPlayer::EditorPlayer()
ItemPlacerMesh =
↪  CreateDefaultSubobject<UStaticMeshComponent>(TEXT("Item
↪  Mesh"));
ItemPlacerMesh->SetCollisionProfileName(FName("ItemPlacer"));
ItemTool->FurnishingMesh = ItemPlacerMesh;

// FurnishingMesh is a pointer in ItemTool to ItemPlacerMesh
```

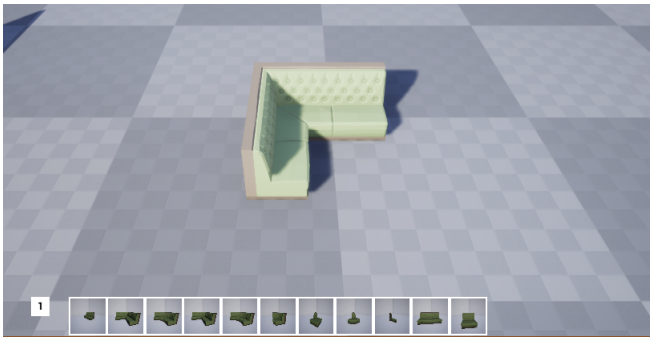

Figure 12. The PlacerMesh in use to place a green sofa in the world

Once a furnishing is selected from the toolbar, the PlacerMesh static mesh component is set to this new mesh and its position is constantly updated in real time via a trace on tick in the Tool. A different furnishing can be selected during this time, which swaps the mesh, and a furnishing can be de-selected by re-selecting the same icon.

```
void UItemTool::TickComponent(float DeltaTime, ELevelTick
↪  TickType, FActorComponentTickFunction* ThisTickFunction)
{
if (bHasItem) {
        // Get Mouse Location in world space
        ToolOwner->PC->DeprojectMousePositionToWorld(MouseWorldLoc,
↪  MouseWorldDirection);
        // Find distance between mouse world pos and closest
↪  floor or placeable object
        bool InteractTrace =
↪  GetWorld()->LineTraceSingleByChannel(TraceHit,
↪  MouseWorldLocation,
    MouseWorldLocation + (MouseWorldDirection *
↪  PlacementDistance),
    ECC_WorldStatic, FCollisionQueryParams(FName("DistTrace"),
↪  true));
        if (bRotateMode == false) {
                FurnishingMesh->SetWorldLocation(TraceHit.Location);
        }
        }
}
```

The player then can place the mesh in the world via the BuildingManager. I also fixed the error requiring the need of an interface, which is now no longer needed.

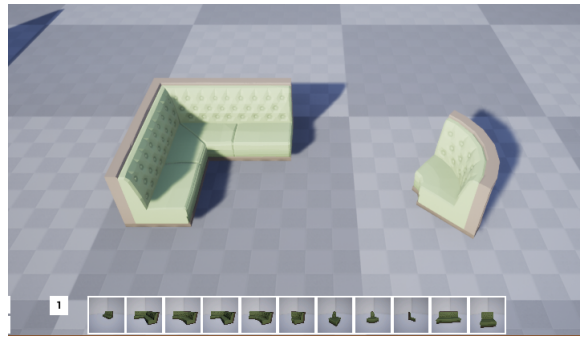

Figure 13. Placing a furnishing. The sofa (left) has been placed in the world, while the new sofa (right) is the PlacerMesh

The Item Tools secondary function is to swap to a rotation mode. When this mode is selected, it locks the PlacerMesh in place and allows the player to rotate the mesh with their mouse, which rotates based on its angle between the mesh and the mouse.

```
}
else if (bRotateMode == true) {
        FRotator rotToMouse =
↪  UKismetMathLibrary::FindLookAtRotation(FurnishingMesh->
    GetComponentLocation(), TraceHit.Location);
        FurnishingMesh->SetWorldRotation(FRotator(0.0f,
↪  rotToMouse.Yaw, 0.0f));
    }
}
```

11 Building Storage

Next, I updated the BuildingManager's method of storing the building data. As the building can consist of multiple floors, I added a new struct to store all of the objects on each floor (walls, furnishings, floors, ceilings), then stored them in a map with an int for the index.

```
USTRUCT(BlueprintType)
struct FBuildingFloorData
{
        GENERATED_USTRUCT_BODY();

public:
//UPROPERTY(EditAnywhere, BlueprintReadWrite)
// WallParent

//UPROPERTY(EditAnywhere, BlueprintReadWrite)
// CeilingParent

//UPROPERTY(EditAnywhere, BlueprintReadWrite)
// FloorParent
```

```
UPROPERTY(EditAnywhere, BlueprintReadWrite)
        TArray<AParentFurnishing*> Furnishings;

UPROPERTY(EditAnywhere, BlueprintReadWrite)
        TArray<FWorldFloorData> FloorGrid;
};
```

Whenever we want to access an item on a specific floor, we can easily find it by only searching through the struct of the floor that it is on. I also updated the furnishing parent to store the objects floor. I also implemented a struct for specifying the size of the world through a grid system, which will be used later on. It is also used to create different levels, with a property defining how large the building can be.

```
USTRUCT(BlueprintType)
struct FWorldFloorData
{
        GENERATED_USTRUCT_BODY();

public:
UPROPERTY(EditAnywhere, BlueprintReadWrite)
        int GridX;

UPROPERTY(EditAnywhere, BlueprintReadWrite)
        int GridY;
};

USTRUCT(BlueprintType)
struct FLevelData : public FTableRowBase
{
GENERATED_USTRUCT_BODY();

public:
UPROPERTY(EditAnywhere, BlueprintReadWrite)
        FString Name;

PROPERTY(EditAnywhere, BlueprintReadWrite)
        TArray<FWorldFloorData> BaseFloorGrid;
};
```

12 Building Grid Ver 1